

Chunking Strategies for Retrieval-Augmented Generation

Introduction

Chunking is the process of dividing a source document into the units that are actually embedded, indexed, and retrieved, and the boundaries chosen during chunking have a larger effect on retrieval quality than many teams initially expect, because a chunk boundary that falls in the wrong place can separate a claim from the evidence that supports it long before the embedding model or the reranker ever gets a chance to see that evidence together.

Unlike embedding model selection or reranker selection, which are usually made once and then held fixed, chunking decisions interact with the structure of every individual source document, so a strategy that works well for narrative prose can behave very differently when applied to a document dominated by tables, nested lists, or deeply nested headings.

This document describes the chunking strategies used in production, the rationale for a roughly five-hundred-token target chunk size, how tables are handled separately from prose before chunking occurs, and the failure modes that chunking strategy choices introduce when a document's structure does not match the assumptions a given strategy was designed around.

Fixed-Size, Structural, and Semantic Chunking Compared

Fixed-size chunking splits a document into chunks of a predetermined token count, typically with no awareness of sentence or paragraph boundaries, which makes it simple and fast to implement but means that a chunk boundary can fall in the middle of a sentence, severing a qualifying clause from the claim it modifies, or can combine the tail of one topic with the head of an unrelated one purely because both happened to fall within the same token window.

Structural chunking instead respects the document's own authored boundaries, such as headings, paragraph breaks, and list item boundaries, grouping structural units together until a size limit is reached and then starting a new chunk at the next convenient structural boundary rather than mid-sentence, which produces chunks that are always coherent at the sentence and paragraph level but does not by itself guarantee that the chunk's content is topically coherent, since an author's paragraph breaks do not always align with a true shift in subject matter.

Semantic chunking goes one step further than structural chunking by using the embedding model itself to decide where topic shifts actually occur: adjacent paragraphs within the same section are embedded, and the similarity between consecutive paragraphs is used to decide whether they belong in the same chunk or whether a new chunk should begin, so that a chunk boundary is placed where the content itself changes topic rather than purely where the author happened to insert a paragraph break.

The production pipeline in this system uses a combination of these ideas rather than any single one in isolation: structural units (headings, paragraphs, lists, tables) are extracted first so that chunk boundaries never fall mid-sentence, adjacent prose units within the same section are then grouped using semantic similarity between consecutive units, and a final size-based pass enforces the target chunk budget on top of whatever the semantic grouping produced, splitting an over-large semantic group or merging adjacent small ones as needed.

A chunk that is too small relative to the target budget carries a different risk than a chunk that is too large: an undersized chunk may lack the surrounding context needed to make its content self-sufficient as a unit of retrieval, forcing the embedding model to summarize a sentence or two in relative isolation, while an oversized

chunk risks diluting a specific relevant detail inside a vector that is forced to represent several unrelated ideas at once, which was the same asymmetry described from the embedding model's perspective in the discussion of chunk design.

The Five-Hundred-Token Target Budget

The production chunking pipeline targets approximately five hundred tokens per chunk, a figure chosen as a balance between two competing pressures: chunks need to be large enough to contain a complete, self-sufficient unit of meaning including any qualifying context, while remaining small enough that a single chunk's embedding is not forced to represent multiple unrelated sub-topics as a single diffuse vector.

Five hundred tokens is also comfortably under the maximum input length of the embedding model used in production, so the target budget is chosen with margin against truncation rather than pushed up against the model's hard limit, since a chunk that occasionally runs slightly over budget due to an indivisible structural unit should still be encoded in full rather than silently truncated.

The five-hundred-token figure is a target rather than a hard ceiling, because some structural units, particularly tables and long list blocks, cannot always be split at an arbitrary token boundary without destroying their meaning, so the size enforcement pass is permitted to produce a chunk somewhat larger than the nominal target when the alternative would be cutting a unit apart in a way that breaks its internal structure.

Chunks that are substantially undersized, for example a single short paragraph left over after a size-based split, are merged with an adjacent chunk from the same section rather than retrieved as an isolated fragment, because a very small chunk is disproportionately likely to be retrieved based on a narrow keyword match rather than genuine topical relevance, given how little surrounding context its embedding has to draw on.

Semantic Shredding: Grouping by Adjacent Similarity

The semantic grouping step compares only consecutive prose units within the same section to one another, rather than performing any form of global clustering across the whole document, which keeps the computation proportional to document length and keeps the resulting chunk boundaries predictable: a chunk boundary only ever appears between two specific adjacent units whose similarity fell below the configured threshold, never as the output of a document-wide optimization that could produce boundaries anywhere.

When the similarity between a unit and the unit immediately preceding it is at or above the threshold, the two are merged into the same growing chunk; when the similarity falls below the threshold, the current chunk is closed off and a new one begins with the unit that triggered the break, so the chunk boundary always lands exactly where the local topic shift was detected rather than at a fixed interval.

Because this grouping operates only within a single section, a heading always forces a chunk boundary regardless of the similarity score on either side of it, which means a chunk never silently spans two differently titled sections even if the embedding model would otherwise judge the content on both sides of the heading to be highly similar.

A table is also never merged into a semantic chunk alongside surrounding prose; tables are treated as a structurally distinct unit type and chunked according to the table-specific rules described later in this document, which keeps a prose explanation and the table it refers to as separate retrievable chunks rather than one combined unit, with the trade-off that a query relying on both the explanation and the table values may need to retrieve two chunks rather than one.

Section Hierarchy and Metadata Propagation

Every chunk retains the hierarchical section path it was extracted from, built by tracking a stack of currently open headings as the document is processed from top to bottom, so a chunk nested under a sub-subsection carries the full path of headings above it rather than only the immediately enclosing heading.

This section path is not used as a substitute for semantic retrieval, meaning the vector search stage does not filter or boost candidates based on section path matching a query, but it is retained as metadata because it gives a human reviewer, or a later analysis pass, the ability to see which part of a document's structure a retrieved chunk actually came from without needing to re-open the source document.

Because section path is derived purely from heading nesting at ingestion time, two differently worded headings about the same underlying concept will not automatically be recognized as related by the chunking pipeline itself; any such connection has to be made by the embedding model at retrieval time based on the chunk's actual textual content, not by the section path metadata.

Table Fragmentation Before Chunking

Tables require handling that is fundamentally different from prose chunking, because a table's rows are only meaningful in relation to its header row, so naively splitting a long table at an arbitrary token boundary, the same way fixed-size chunking would split prose, would produce a second fragment whose rows have no header context at all once that fragment is retrieved on its own.

The production pipeline addresses this by fragmenting large tables before chunking rather than after: if a table's full markdown serialization exceeds the configured token budget for tables, the table is split into multiple fragments by rows, and the header row is repeated at the top of every fragment, so each fragment remains independently interpretable even if only one fragment out of several is ever retrieved for a given query.

A table that is small enough to fit within its token budget as a single unit is never fragmented at all and is stored as exactly one chunk containing the complete header and every row, which means small comparison tables, the kind used throughout this corpus, typically survive ingestion as a single retrievable unit rather than being split across several chunks.

Table fragmentation happens before the general chunking pass specifically so that the token-budget enforcement applied to prose chunks never has the opportunity to see a table as an undifferentiated block of text and split it at a row boundary that happens to fall in an arbitrary place; by the time the general chunking logic runs, every table has already been resolved into one or more already-correctly-sized, header-preserving fragments.

Overlap Strategies and Their Trade-offs

Some chunking designs deliberately repeat a small amount of content at the boundary between two adjacent chunks, commonly called sliding-window overlap, so that a sentence sitting exactly at a chunk boundary appears fully within at least one of the two chunks rather than being split across both.

Overlap reduces the risk that a boundary falls in an unfortunate place relative to a specific piece of evidence, but it does so at the cost of inflating the total number of stored chunks and reintroducing a version of the near-duplicate content problem, where two chunks that mostly repeat each other's content can both surface as separate, redundant entries in a candidate pool, displacing a chunk from a different part of the document that could have offered additional distinct evidence.

The production pipeline does not use sliding-window overlap, relying instead on semantic grouping to place chunk boundaries at genuine topic shifts rather than at arbitrary token counts, on the reasoning that a boundary chosen because the content actually changed topic at that point is less likely to need overlap protection than a boundary chosen purely because a fixed token count was reached.

This design choice does mean that a chunk boundary occasionally still separates two sentences that a human reader would consider closely related, particularly when a semantic similarity score sits right at the configured threshold, and no overlap mechanism exists to mitigate that specific case; the mitigation relied upon instead is that the embedding model is expected to place both resulting chunks close enough together in vector space that both are likely to surface among the retrieved candidates even when split.

Chunk Size and Retrieval Quality

Internal measurements comparing different fixed target chunk sizes against the same evaluation query set found that very small chunks and very large chunks both underperform a mid-range target, though for different underlying reasons, which is summarized in the following table of illustrative figures from an internal chunk-size study on a stable corpus:

Target chunk size (tokens)	Mean chunks per document	Recall@10	Precision@10	Dominant failure mode
150	41	0.52	0.09	keyword-sensitive, context-poor chunks
300	24	0.61	0.12	some context loss at boundaries
500	15	0.64	0.14	best balance observed
800	9	0.58	0.13	diffuse embeddings, mixed sub-topics
1200	6	0.49	0.11	severe topic dilution per chunk

The recall decline at both extremes of this table reflects two distinct mechanisms: small chunks lose recall because their embeddings are overly sensitive to whichever handful of terms happen to appear in a short span, causing genuinely relevant but differently worded chunks to rank lower than keyword-overlapping but less relevant ones, while large chunks lose recall because a single chunk's embedding is forced to represent multiple sub-topics simultaneously, diluting the specific signal a query is looking for.

Precision follows a similar but not identical pattern, peaking at the five-hundred-token target and declining at both extremes, though the decline in precision is generally smaller in magnitude than the decline in recall, suggesting that chunk size primarily affects whether relevant material is surfaced at all rather than how cleanly irrelevant material is excluded once relevant material is present in the candidate pool.

Failure Modes

Boundary splitting mid-argument is the most direct failure mode: even with semantic grouping in place, a chunk boundary can fall between a claim and the specific caveat or exception that qualifies it, if the similarity between the two adjacent paragraphs happened to fall just below the merging threshold, which produces a retrieved

chunk that is technically accurate but incomplete, stating a claim without its necessary qualification.

Orphaned headings occur when a heading unit ends up alone in its own chunk, separated from the content that follows it, typically because a size-based split occurred immediately after the heading rather than after at least one paragraph of body text, producing a retrievable chunk whose entire content is a heading title with no supporting prose, which carries almost no useful retrieval signal on its own.

Context fragmentation describes the broader pattern where a single coherent explanation spanning several paragraphs is split across multiple chunks by a size-based cutoff landing in the middle of that explanation, so that a query whose answer depends on the explanation's full arc may only retrieve part of it, and the system must rely on multiple chunks being retrieved together, with no mechanism that recombines them before reranking, to reconstruct the complete picture.

Table-adjacent paragraph separation is a failure mode specific to the table-handling design: because tables are chunked independently from the prose immediately preceding or following them, a paragraph that introduces a table, explaining what its columns mean or what conclusion to draw from it, is retrieved as a separate chunk from the table itself, so a query that would be best answered by the paragraph and the table together instead depends on both chunks being retrieved and reranked as if they were independent pieces of evidence.

Practical Implementation Considerations

Authoring source documents with deliberate paragraph breaks at genuine topic shifts, rather than breaking paragraphs purely for visual formatting reasons, measurably improves how well semantic grouping places chunk boundaries, since the grouping logic can only ever place a boundary between two existing structural units, never in the middle of one.

Keeping individual tables reasonably small, rather than constructing a single sprawling table that mixes several unrelated comparisons into one structure, avoids unnecessary table fragmentation and keeps a table's full context, including its header row, available in a single retrievable chunk rather than split across fragments.

Heading granularity also matters operationally: a document with very few headings forces semantic grouping to do most of the work of finding topic boundaries within long, loosely structured sections, while a document with excessively fine-grained headings, one for every single paragraph, forces nearly every paragraph into its own chunk regardless of whether adjacent paragraphs are actually on the same topic, since a heading always forces a chunk boundary.

List and Enumeration Handling During Chunking

Each list item is extracted during structural extraction as its own individual structural unit, distinct from its sibling items in the same list, which means a bulleted or numbered list of several items is represented internally as a sequence of separate units rather than as one combined list block, and these individual list-item units then participate in semantic grouping exactly the way individual paragraphs do.

This item-level granularity has a specific consequence for chunking: if the similarity between two consecutive list items happens to fall below the semantic grouping threshold, a single list can be split across two separate chunks partway through its own enumeration, producing one chunk that contains only the first few items of a list with no indication, from that chunk's content alone, that additional items exist elsewhere in a sibling chunk. A single list item retrieved in isolation, separated from the heading or introductory sentence that established what the list is actually enumerating, can be a particularly weak piece of retrievable evidence, since an isolated bullet such as a short phrase describing one failure mode or one configuration option carries much less self-sufficient context than an equivalent sentence embedded in ordinary prose would, because the bullet's embedding has no surrounding sentence structure to draw additional context from.

This is a structural trade-off rather than an oversight: treating list items individually, rather than always merging an entire list into one combined chunk regardless of size, allows a very long list to still be split sensibly across multiple appropriately sized chunks, at the acknowledged cost that any individual list item chunk may carry less standalone context than a prose paragraph chunk of comparable length would.

Re-Chunking After Source Document Edits

Because chunking operates on the complete structural extraction of a document, there is no mechanism to selectively re-chunk only the specific part of a document that changed; any edit to a source document, however small, requires the entire document to be re-ingested and re-chunked from scratch as a new current version under its existing logical document identifier.

A consequence of this whole-document re-chunking is that chunk boundaries are not guaranteed to remain stable across re-ingestion even in sections of the document that were not directly edited, since semantic grouping and size-based splitting both operate on the document's content as a whole, so a change early in a long document can, in principle, shift where later chunk boundaries fall simply because the token accounting and similarity comparisons that produced the previous chunk boundaries are recomputed fresh against the updated full document.

This means chunk identifiers themselves are not stable identifiers for a specific piece of content across different versions of a document; a given fact that lived in one specific chunk in an earlier version is not

guaranteed to live in a chunk with any particular relationship to its previous chunk once the document has been re-ingested, which is one of the reasons the system's versioning model treats an entire re-ingested document as producing an entirely new, independent set of chunks rather than attempting to track continuity of individual chunks across versions.

Minimum Content Thresholds

A structural unit, or a small group of merged units, that falls below a minimum content threshold even after the merge-with-adjacent-chunk step described earlier is applied, for example a single short list item left trailing at the very end of a section with no further content after it to merge into, is discarded entirely rather than persisted as an extremely small, low-information chunk, on the reasoning that a chunk carrying only a handful of tokens is more likely to introduce noise into the candidate pool through spurious keyword matches than to ever serve as genuinely useful retrieval evidence on its own.

This discard behavior is distinct from the earlier-described merging behavior: merging is attempted first, combining an undersized trailing unit with its immediately preceding chunk in the same section, and only once no adjacent chunk is available to merge into, which happens at a section boundary, does the undersized unit get dropped rather than persisted as an isolated minimal chunk.

A heading unit that ends up with no body content following it anywhere before the next heading or the end of the document, the orphaned-heading case described elsewhere in this corpus, is one concrete example that can trigger this discard path, since a lone heading with nothing to merge into on either side has no substantial content to contribute as a standalone chunk in the first place.

Limitations

Chunking decisions are made once, at ingestion time, and are not reconsidered per query; there is no mechanism in this pipeline that dynamically recombines or re-splits chunks based on what a specific incoming query is asking about, so the chunk boundaries chosen at ingestion time apply uniformly to every future query against that document regardless of how well or poorly those particular boundaries suit any individual query's needs.

Because chunking operates purely on document structure and adjacent-unit semantic similarity, it has no awareness of the evaluation queries that will eventually be run against the corpus, and a corpus author cannot guarantee in advance that a specific piece of evidence will land in exactly one clean, self-sufficient chunk; the quality of chunk boundaries is a statistical property of the pipeline applied to a given document's structure, not a guarantee made for any individual fact.